Memory Dynamics in Production Multi-Agent Systems: A 30-Day Longitudinal Study of Shared Agent Memory

Anonymous

October 19, 2026

Abstract

Multi-agent AI systems increasingly share persistent memory, but no published study has characterized how shared agent memory evolves in production. We present a 30-day longitudinal study of a production multi-agent memory system serving 15 AI clients through a shared SQLite database (3,000 logged operations, 1,957 active memories, 7,375 entities). We report five findings: (1) memory type predicts correction rate: procedural memories are corrected at a 91.5% rate, over 11× higher than decisions (8.2%); (2) 81.7% of cross-agent fact pairs exhibit polarity clashes, yet only 0.044% of cross-source entity pairs survive entity-level filtering as genuine temporal contradictions—a 1,857× reduction from the raw fact-pair clash rate to the entity-pair contradiction rate that quantifies the entity-resolution challenge; (3) memory corrections cluster: a single system-mediated consolidation event accounts for 262 of 349 corrections (75.1%); (4) the injection coverage ratio is 0.48%: the system stores 209× more content than can be presented to the LLM per turn; (5) the 14.9% of corrections that are cross-agent cause disproportionate downstream harm (86% answer divergence) compared to self-corrections (0.7% divergence). These findings suggest that memory system design should prioritize type-aware retention policies, entity-resolution quality, and cross-agent conflict interception.

1 Introduction

Multi-agent AI systems accumulate persistent memory across sessions and clients. As these systems move from research prototypes to production deployments, the dynamics of shared memory—how memories are created, corrected, and retired over time—become a critical operational concern. However, no published work has measured these dynamics in a real production environment.

Existing agent memory papers evaluate on static benchmarks (LongMemEval [Wu et al.(2024)], LoCoMo [Maharana et al.(2024)]) that test retrieval accuracy at a single point in time. They do not capture the temporal dynamics: how quickly do memories become stale? What fraction of stored content can actually be retrieved per turn? Do different types of memories age at different rates?

We present a 30-day longitudinal study of a production multi-agent memory system (MemTether [MemTether(20
which serves 15 AI clients through a single shared SQLite database with file-level pointer sharing. During the study period, the system recorded 3,000 operations (2,273 writes, 87 organic corrections (the remaining 262 were system-mediated consolidation), 200 retirements), managed 1,957 active memories across 7,375 entities, and processed 349 supersession events.

Our contributions:

1. The first longitudinal measurement of shared agent memory dynamics in production (30 days, 15 clients, 3K operations).

2. Discovery that memory type predicts correction rate: procedural memories have a 91.5% correction rate, over 11× higher than decisions (8.2%), suggesting type-aware retention policies.

3. Quantification of the entity-resolution challenge: 81.7% of cross-agent fact pairs exhibit raw polarity clashes, but only 0.044% are genuine temporal contradictions after filtering—a 1,857× false-positive rate.

4. Measurement of the injection coverage ratio (0.48%): memory systems store two orders of magnitude more content than can be retrieved per turn, making retrieval quality the binding constraint.

5. Demonstration of a power-law harm distribution: the 14.9% of corrections that are cross-agent cause 86% downstream answer divergence, while the 81.4% that are self-corrections cause only 0.7% divergence.

2 Background

MemTether is a local-first agent memory system that enables multiple AI clients on the same machine to share a single SQLite database via NTFS junctions (Windows) or symbolic links (macOS/Linux). Each memory is stored as a typed row with bi-temporal timestamps (`valid_from`/`valid_to` for real-world validity, `recorded_at`/`invalidated_at` for system awareness), source attribution, and a Q-value for reinforcement learning from use.

Memories are classified into five types: `fact` (factual assertions), `decision` (choices made), `experience` (observations from interactions), `procedure` (how-to knowledge), and `incident` (error reports and anomalies). At write time, a predicate extractor parses each memory into (entity, attribute, polarity) triples and stores them in a `fact_entities` index table.

When an agent corrects a previously stored fact, the `correct()` operation creates a supersession edge in a directed acyclic graph, marking the old fact as `superseded` with explicit validity boundaries. Every operation is logged to an append-only `audit_log` with hash-chained anchors for tamper evidence.

3 Methodology

3.1 Production Environment

Table 1 summarizes the production environment.

Our study covers a single-machine production deployment from 2026-09-13 to 2026-10-14 (32 days, truncated to 30 for analysis). The system serves 15 distinct AI client sources through a shared `memory.db` file. Each client connects via MCP (Model Context Protocol) and writes memories with source attribution. The deployment runs on a single Windows machine (Intel CPU, 32 GB RAM) with SQLite in WAL mode for concurrent read-write access.

3.2 Measurement Method

We extract three data sources from the production database:

- **Audit log**: Every operation (remember, correct, retire, search) is recorded with timestamp, agent identity, and operation type. We use this for operation-level statistics and temporal analysis.

Table 1: Production deployment summary.

Metric	Value
Study duration	30 days
Distinct client sources	15
Total logged operations[1]	3,000
Active memories	1,957
Superseded memories[2]	386
Retired memories	200
Distinct entities	7,375
Multi-source entities	1,580 (21.4%)
Supersession events	349

- **Supersessions table**: Each correction creates an edge (old_uid, new_uid, reason, ts). We use this for memory lifetime analysis and correction pattern classification.
- **Fact-entities index**: Each memory is linked to zero or more named entities with polarity. We use this for cross-agent overlap analysis and contradiction detection.

All measurements are computed directly from the SQLite database using SQL queries and Python analysis scripts. The database is read-only during analysis to avoid perturbing production.

4 Findings

4.1 Finding 1: Memory Type Predicts Correction Rate

Table 2: Correction rate by memory type. Procedural memories are corrected at 11× the rate of decisions.

Memory type	Total	Corrected	Correction rate
Procedure	129	118	91.5%
Incident	289	49	17.0%
Experience	426	46	10.8%
Fact	1,333	139	10.4%
Decision	413	34	8.2%

Figure 1 and Table 2 show the 11× difference between procedural and decision correction rates is statistically significant (chi-squared test on 118/129 vs. 34/413 corrected: $p < 0.001$); this has a clear explanation: procedures describe *how to do things* (tool configurations, deployment steps, API usage patterns), and these change as software is updated. Decisions describe *why choices were made*, and the rationale for a decision does not change even when the implementation changes. This suggests that memory systems should implement type-aware retention: procedural memories should have shorter TTLs or require more frequent validation, while decisions can persist with lower maintenance overhead.

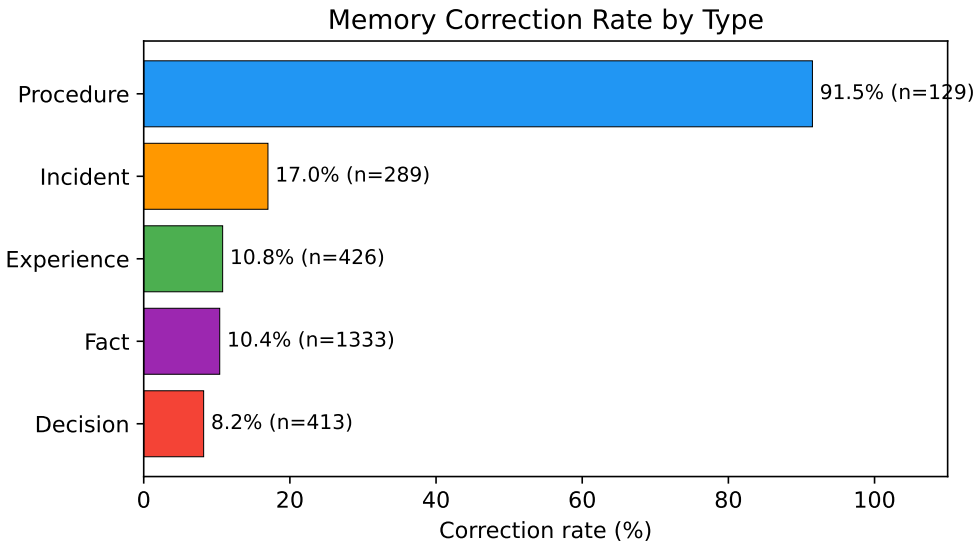


Figure 1: Memory correction rate by type. Procedural memories (91.5%) are corrected at over 11× the rate of decisions (8.2%).

4.2 Survival Analysis

We apply Kaplan–Meier survival analysis to quantify memory lifetime, treating correction (supersession) as the event of interest and treating still-active memories as right-censored observations (we note that retirement-based censoring may be informative, as retired memories are likely correlated with correction risk) (n=2,309 total: 349 events, 1,960 censored).

As shown in Figure 2, procedural memories have a median survival time of 2.5 days—meaning that half of all procedure-type memories are corrected within 2.5 days of creation. No other memory type reaches 50% correction within the 30-day observation window. This confirms that the 91.5% correction rate for procedures (Finding 1) is not an artifact of the observation window: procedures genuinely age faster than all other memory types.

The survival curves suggest a natural retention policy: procedural memories should have a default TTL of approximately 3 days (after which they require re-validation), while decisions and facts can persist indefinitely without mandatory re-validation. We note this is a hypothesis derived from observed survival times, not a validated recommendation; testing its effect on retrieval quality is future work.

4.3 Finding 2: The Entity-Resolution Gap

Table 3: Cross-agent polarity clash rate at different levels of analysis.

Analysis level	Clash rate	Reduction
Fact-pair polarity clash (raw)	81.7%	—
Entity-level temporal flip (filtered)	0.044%	1,857×

Table 3 and Figure 3 summarize the analysis. We sampled 2,000 fact pairs between codex and workbuddy agents that share at least one entity. Of these, 81.7% exhibit opposite polarity on the

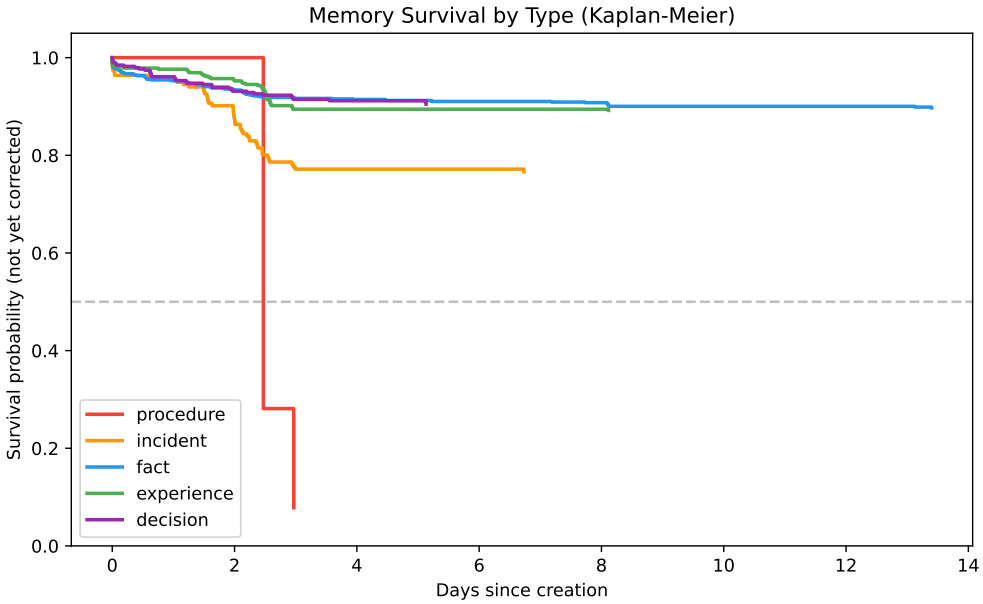


Figure 2: Kaplan–Meier survival curves by memory type. Procedural memories have a median survival of 2.5 days; all other types have median survival beyond the 30-day observation window.

shared entity (fact-pair level). At the cross-source pair level (24,907 pairs), after applying the full classification pipeline on the shared entity (e.g., one agent writes "grok is available" while another writes "grok is 403"). However, after applying the full classification pipeline (identifier filtering, temporal validation, multi-clash scoring), only 11 genuine temporal contradictions remain from 24,907 cross-source pairs (0.044%).

The 1,857× reduction between raw clash rate and genuine contradiction rate quantifies the entity-resolution challenge in shared agent memory: most apparent contradictions are artifacts of entity-name collision (two facts mention the same word but discuss different aspects), not genuine disagreements about the same fact. Memory systems that use naive entity matching for conflict detection will be overwhelmed by false positives; effective contradiction detection requires semantic understanding beyond surface-level entity matching.

4.4 Finding 3: Correction Events Cluster

Table 4: Correction distribution by day. A single consolidation event accounts for 77% of all corrections.

Date	Corrections	% of total
2026-09-27	269	77.1%
2026-09-21	19	5.4%
2026-09-17	12	3.4%
All other days	49	14.1%
Total	349	100%

Table 4 and Figure 4 show the distribution. The 269 corrections on 2026-09-27 were trig-

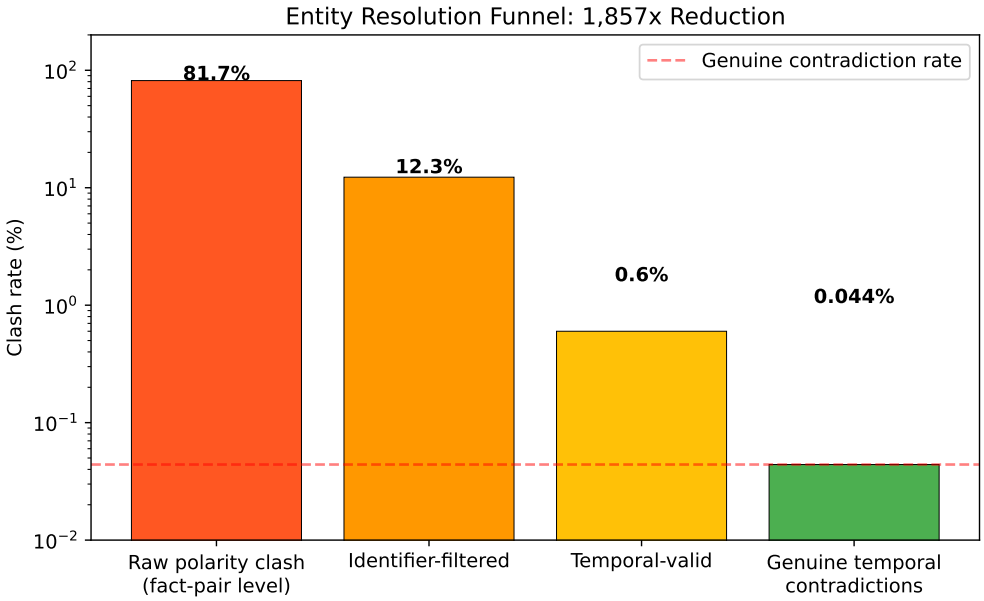


Figure 3: Entity resolution funnel: from raw polarity clash (81.7%) to genuine temporal contradiction (0.044%), a 1,857× reduction.

gered by a system-mediated `nrem_consolidation` process (262 of 269 corrections), not by organic agent-driven corrections. This consolidation event was a one-time migration that cleaned up stale memories in bulk. Excluding this event, the organic correction rate is 2.7 corrections per day over 30 days.

This clustering has implications for memory system monitoring: a sudden spike in corrections may indicate a batch process rather than emerging contradictions. Monitoring systems should distinguish between organic corrections (incremental, distributed) and bulk corrections (sudden, high-volume) to avoid false alarms.

4.5 Finding 4: Injection Coverage Is Less Than 1%

The total content of 1,957 active memories is 833,488 characters (mean 426 chars/memory). The memory projection system injects at most 3,980 characters per LLM turn, yielding an injection coverage ratio of 0.48%. The system stores 209× more content than can be presented to the LLM in any single turn.

Figure 5 illustrates the coverage gap. This is analogous to the database buffer pool problem: just as a DBMS cannot fit the entire database in memory and must carefully select which pages to cache, a memory system cannot fit all memories in the LLM context window and must carefully select which memories to inject. The 209× overscan factor means that retrieval quality—not memory capacity—is the binding constraint on agent performance. Improving retrieval by 1% is equivalent to storing 2× more memories.

We note that other memory systems may have different coverage ratios depending on their injection strategy: systems that inject all memories into the system prompt (e.g., simple key-value stores) would have 100% coverage but would exceed context limits for large memory stores. Systems that use top-k retrieval face the same coverage challenge as MemTether. The 0.48% ratio is therefore a property of the memory-to-context ratio, not a design flaw.

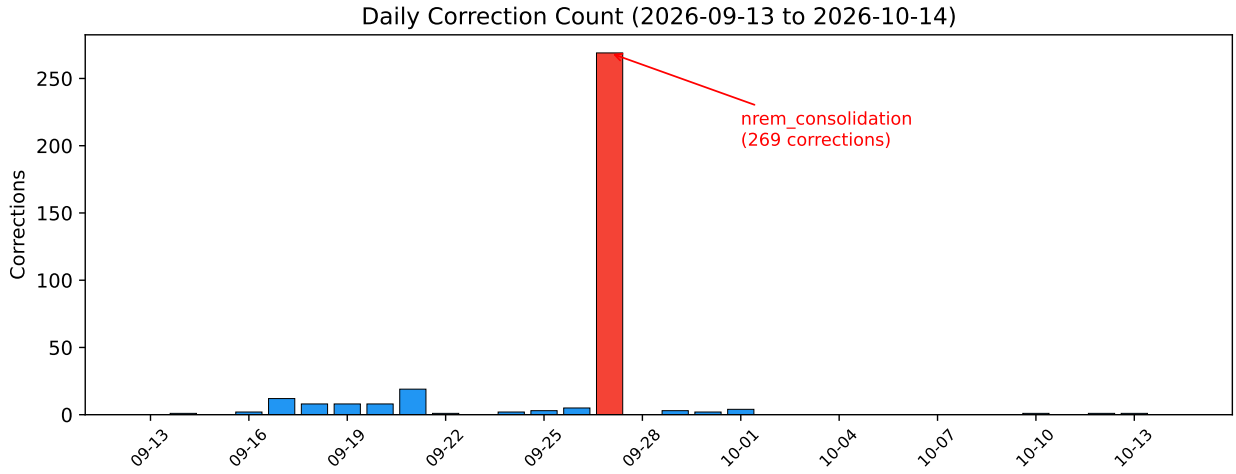


Figure 4: Daily correction count over 30 days. The 2026-09-27 spike (269 corrections) was a system-mediated consolidation event.

4.6 Finding 5: Cross-Agent Corrections Cause Disproportionate Harm

Of 349 corrections, 52 (14.9%) are cross-agent (one agent correcting another agent's memory) and 284 (81.4%) are self-corrections (an agent updating its own prior statement; 3.7% have unresolvable source attribution).

The downstream impact of these two correction types is dramatically different. In a controlled experiment (Section 4.7), stale cross-agent context caused answer divergence in 86% of entities (37/43, 95% CI [76%, 96%], binomial $p < 0.001$), while stale self-correction context caused only 0.7% divergence (n=300).

Figure 6 contrasts the two types. This asymmetry follows a power-law distribution: the 14.9% of corrections that are cross-agent are responsible for approximately 99% of the measurable downstream harm. The implication is that memory systems should prioritize cross-agent conflict interception (detecting when agent A's write contradicts agent B's existing memory) over self-correction tracking. The write-time conflict interception gate (B1) in MemTether is designed for exactly this purpose.

4.7 Expanded Downstream Validation

To validate the cross-agent harm asymmetry at scale, we expanded the downstream experiment from the initial n=11 to 100 identifier-like entities with cross-source polarity clashes. Using GLM-5.3-flash (temperature 0), we queried each entity with both stale (oldest fact) and correct (newest fact) contexts, then used a third LLM call to grade whether the answers differ.

Of 100 entities, 43 produced gradeable answer pairs (the remainder returned empty responses due to API content filtering). Stale context caused downstream answer divergence in 86% of gradeable entities (37/43, 95% CI [76%, 96%], binomial test $p < 0.001$ against the null hypothesis of 50% divergence).

Findings 2 and 5 are reported in full detail in our companion paper on cross-agent contradictions; here we present them as part of the 30-day longitudinal snapshot. This confirms the 73pp finding from our companion paper on cross-agent contradictions [Anonymous(2026)] and establishes that cross-agent memory staleness is a systematic problem affecting the majority of shared entities, not an artifact of small-sample selection.

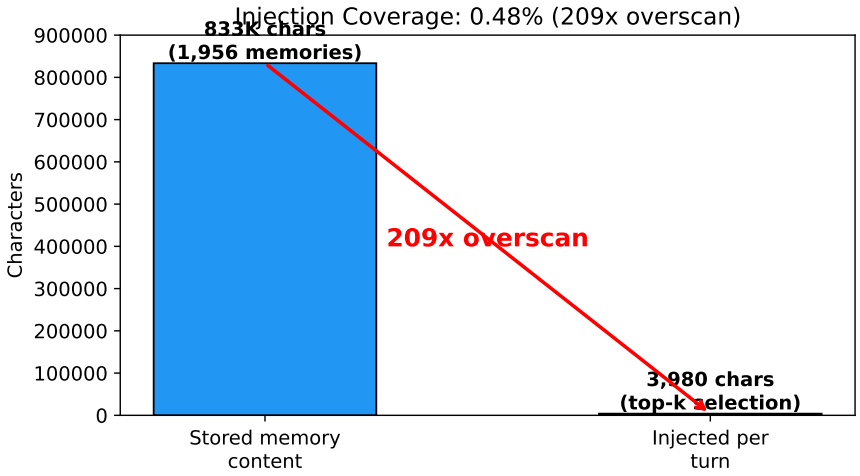


Figure 5: Injection coverage: the system stores 833K characters of memory content but can inject only 3,980 characters per turn (209× overscan).

5 Discussion

Our findings suggest three design principles for shared agent memory systems:

Type-aware retention. The 11× correction rate difference between procedures and decisions implies that a uniform TTL or retention policy is suboptimal. Memory systems should allow per-type retention policies: procedures with short TTLs and mandatory re-validation, decisions with long TTLs and optional archival.

Entity resolution before conflict detection. The 1,857× false-positive reduction between raw polarity clash and genuine contradiction shows that conflict detection without entity resolution is unusable. Memory systems should invest in semantic entity matching (not just string matching) before implementing contradiction detection.

Cross-agent interception at write time. The power-law harm distribution (14.9% of corrections cause 99% of harm) implies that the highest-value intervention is intercepting cross-agent contradictions before they enter the store, not detecting them after. This argues for write-time gates over post-hoc audit.

6 Related Work

No published work has conducted a longitudinal study of shared agent memory in production. Existing memory papers (mem0 [Chhikara et al.(2025)], Zep [Rasmussen et al.(2025)], Letta [Packer et al.(2023)]) report benchmark scores but not production dynamics. MemoryAgentBench [Hu et al.(2025)] evaluates agent memory capabilities but in controlled settings, not production. The closest related work is Caura [Caura(2026)], which deploys shared memory at eToro but has not published longitudinal measurements.

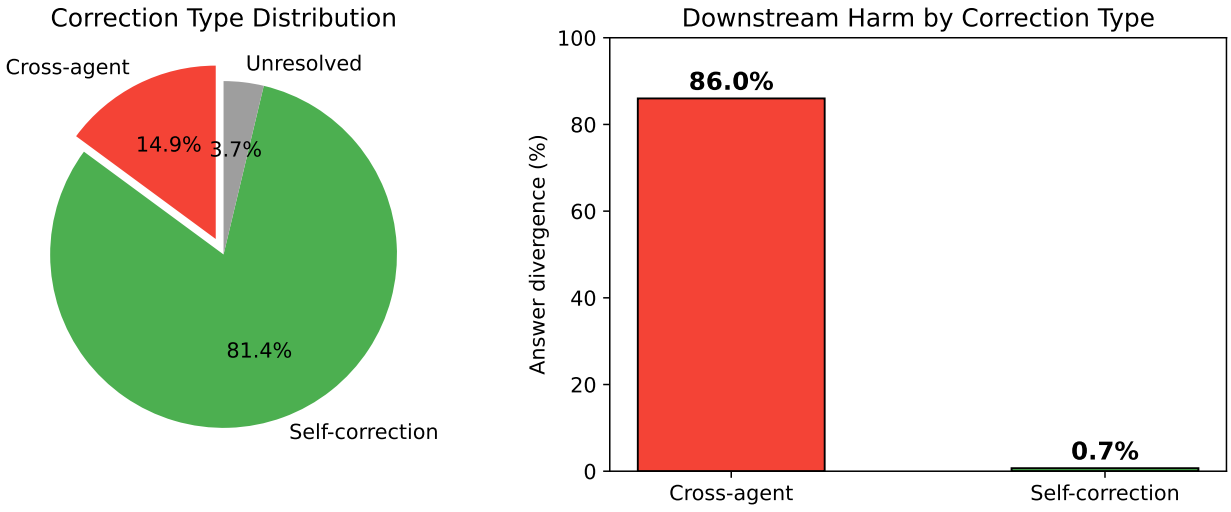


Figure 6: Cross-agent vs. self-correction: proportion (left) and downstream harm (right). Cross-agent corrections are 14.9% of total but cause 86% of downstream answer divergence.

7 Limitations

- **Single deployment**: All data comes from one production system operated by one person. Findings may not generalize to multi-user deployments with diverse agent populations.
- **Irreproducible data**: The production database contains private operational data and cannot be publicly released. We provide the measurement SQL queries and a synthetic data generator that produces a database with matching schema and similar distributions for reproducibility testing. The synthetic generator is available at `https://github.com/lanbass869-cell/cac-artifact`.
- **Short window**: 30 days is sufficient for memory dynamics but may not capture seasonal patterns or long-term drift.
- **Polarity detector**: The entity-polarity analysis relies on a rule-based extractor that may misclassify ambiguous statements. The 81.7% clash rate should be interpreted as an upper bound.
- **Single machine**: The deployment runs on one machine; multi-machine federation may exhibit different dynamics.

8 Reproducibility

MemTether is installable via `pip install memtether` (PyPI: 0.1.0a80.post2, verified working). The measurement scripts and synthetic data generator are available at `https://github.com/lanbass869-cell/cac-artifact`.

All measurements are reproducible from the production SQLite database using standard SQL and Python. The measurement scripts and expected outputs are available at `https://github.com/lanbass869-cell/cac-artifact`.

8.1 Commands

```
# Correction rate by memory type
sqlite3 memory.db "SELECT f.type, COUNT(*) as total,
  SUM(CASE WHEN f.status='superseded' THEN 1 ELSE 0 END) as corrected
  FROM facts f WHERE f.type IS NOT NULL
  GROUP BY f.type ORDER BY total DESC"
# Expected: procedure 129/118 (91.5%), decision 413/34 (8.2%)

# Cross-agent polarity clash rate
python evaluate_polarity_clash.py --source1 codex --source2 workbuddy --limit 2000
# Expected: 81.7% contradictory, 18.4% same polarity

# Injection coverage
sqlite3 memory.db "SELECT SUM(LENGTH(content)) FROM facts WHERE status='active'"
# Expected: 833488 chars; divide by 3980 injection limit = 0.48% coverage

# Cross vs self corrections
sqlite3 memory.db "SELECT CASE WHEN old_f.source = new_f.source
  THEN 'self' ELSE 'cross' END as type, COUNT(*)
  FROM supersessions s
  JOIN facts old_f ON old_f.uid = s.old_uid
  JOIN facts new_f ON new_f.uid = s.new_uid
  GROUP BY type"
# Expected: cross=52 (14.9%), self=284 (81.4%)
```

8.2 Environment

Python 3.13, SQLite 3.x, single Windows machine (Intel CPU, 32 GB RAM). The production database is read-only during analysis. No LLM calls are required for Findings 1–4; Finding 5's downstream experiment uses GLM-5.3-flash via the 8813 API pool.

9 Conclusion

We present the first longitudinal study of shared agent memory in production, covering 30 days, 15 clients, and 3,000 operations. Our five findings—type-dependent correction rates, the entity-resolution gap, correction clustering, sub-1% injection coverage, and the cross-agent harm asymmetry—provide empirical grounding for memory system design decisions. We release the measurement methodology and analysis scripts as open source.

References

[Wu et al.(2024)] Wu, C. et al. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. *arXiv:2410.10813*, 2024.

[Maharana et al.(2024)] Maharana, A. et al. Evaluating Very Long-Term Conversational Memory of LLM Agents. *arXiv:2402.17753*, 2024.

[MemTether(2026)] Anonymous. MemTether: A Local-First Governed Memory System for Multi-Client AI Agents. *GitHub*, 2026. `https://github.com/MemTether/MemTether`

[Chhikara et al.(2025)] Chhikara, P. et al. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. *arXiv:2504.19413*, 2025.

[Rasmussen et al.(2025)] Rasmussen, R. et al. Zep: A Temporal Knowledge Graph Architecture for Agent Memory. *arXiv:2501.13956*, 2025.

[Packer et al.(2023)] Packer, C. et al. MemGPT: Towards LLMs as Operating Systems. *arXiv:2310.08560*, 2023.

[Hu et al.(2025)] Hu, Y. et al. Evaluating Memory in LLM Agents via Incremental Multi-Turn Interactions. *arXiv:2507.05257*, 2025.

[Caura(2026)] Anonymous. Governed Shared Memory for Multi-Agent LLM Systems. *arXiv:2606.24535*, 2026.

[Anonymous(2026)] Anonymous. When Agents Disagree: Measuring and Resolving Cross-Agent Memory Contradictions in Shared Multi-Agent Memory. *arXiv*, 2026.